

인턴 · 초급 개발자 실습 참고자료

Dockerfile & docker-compose 작성법

스프링부트 프로젝트를 기준으로 — 요소 하나하나 설명 + 실행되는 예제

Dockerfile 지시어

멀티스테이지 빌드

compose 키

app + MySQL

실무 함정

기준 예제 · intro-jpa (Spring Boot 4.1 / Gradle / H2-MySQL) | 이 문서의 모든 예제는 실제 docker build · docker compose up 으로 실행 검증됨

시작하며 — 무엇을, 왜

지난 「컨테이너·Docker·쿠버네티스 입문」에서 **개념** 을 잡았다면, 이번엔 **직접 작성**합니다. 다룰 것은 딱 두 파일입니다.

1. **Dockerfile** — 내 스프링부트 앱을 **이미지(붕어빵 틀)**로 굽는 레시피
2. **docker-compose.yml** — 앱 컨테이너와 DB 컨테이너를 **한 번에 띄우는 지휘서**

실행 검증됨

이 문서의 Dockerfile·compose 예제는 눈대중이 아니라 **실제로 빌드·실행해 확인**했습니다: 멀티스테이지 `docker build` 성공(최종 이미지 409MB), `docker compose up` 으로 스프링부트 앱이 **MySQL 컨테이너에 접속(MySQLDialect) → HTTP 200**까지 확인.

기준 예제 프로젝트

회사 교육에서 만든 **intro-jpa**를 그대로 씁니다. 특징만 짚으면:

- Spring Boot **4.1**, Java 17, **Gradle**(래퍼 포함) → 빌드 결과물은 `build/libs/intro-jpa-0.0.1-SNAPSHOT.jar`
- 기본 프로파일은 **H2**(파일), `mysql` 프로파일이면 **MySQL**에 접속 — 이 "프로필 전환"이 compose 예제의 핵심 소재
- 웹 포트 **8080**

준비물

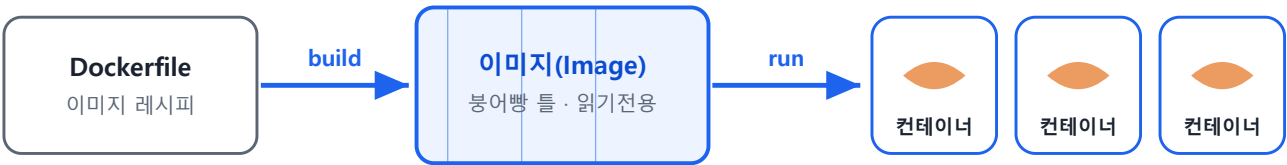
- **Docker Desktop** 설치(윈도우/맥). 설치되면 `docker` 와 `docker compose` 명령을 쓸 수 있습니다.
- 확인: `docker --version` , `docker compose version`

용어 빠른 정리

이미지=실행에 필요한 걸 다 담아 굳힌 읽기전용 꾸러미(붕어빵 틀). **컨테이너**=이미지를 실행한 상태(붕어빵). **Dockerfile**=이미지를 만드는 설명서. **레지스트리**=이미지 저장소(Docker Hub).

1 Dockerfile — 이미지를 굽는 레시피

Dockerfile은 "이 이미지를 어떻게 만들어라"를 위에서 아래로 적은 설명서입니다. `docker build` 가 이 파일을 한 줄씩 읽어 이미지를 만들고, 그 이미지를 `run` 하면 컨테이너가 됩니다.



틀 1개(이미지)로 컨테이너 여러 개를 똑같이 실행

그림 1. Dockerfile → (build) 이미지 → (run) 컨테이너

1-1. 주요 지시어(instruction) 총정리

Dockerfile은 **대문자 지시어 + 인자** 형태의 줄로 이뤄집니다. 자주 쓰는 것만 추리면 아래가 거의 전부입니다.

지시어	하는 일	스프링부트 예시
FROM	베이스 이미지 지정(출발점). 모든 Dockerfile의 첫 줄	FROM eclipse-temurin:17-jre
WORKDIR	이후 명령의 기준 작업 폴더 지정(없으면 만들)	WORKDIR /app
COPY	호스트 파일 → 이미지 안으로 복사	COPY build/libs/*.jar app.jar
ADD	COPY와 비슷하나 URL·압축 해제 기능(특수). 보통 COPY 권장	거의 안 씀
RUN	이미지 빌드 중 명령 실행(결과가 이미지 레이어로 남음)	RUN ./gradlew bootJar
ENV	환경변수 설정(런타임까지 유지)	ENV TZ=Asia/Seoul
ARG	빌드 시점 에만 쓰는 변수(런타임엔 없음)	ARG JAR=build/libs/*.jar
EXPOSE	이 컨테이너가 쓰는 포트 문서화 (실제 개방은 아님)	EXPOSE 8080
ENTRYPOINT	컨테이너 시작 시 항상 실행 되는 고정 명령	ENTRYPOINT ["java","-jar","app.jar"]
CMD	기본 명령/기본 인자 (실행 시 덮어쓰기 쉬움)	CMD ["--server.port=8080"]
USER	실행 사용자 지정(root 대신 → 보안)	USER spring
VOLUME	데이터 보존용 마운트 지점 선언	VOLUME /app/logs
LABEL	이미지 메타데이터 (작성자 등)	LABEL team="backend"

1-2. 헛갈리는 짝: ENTRYPOINT vs CMD

둘 다 "컨테이너가 시작할 때 뭘 실행할지"를 정하지만 역할이 다릅니다.

	ENTRYPOINT	CMD
성격	항상 실행되는 실행 파일	기본값 (기본 명령 또는 기본 인자)
docker run 뒤 인자	ENTRYPOINT의 인자로 덧붙임	통째로 대체됨
함께 쓰면	ENTRYPOINT=실행할 것 (java -jar app.jar)	CMD=기본 인자 (--옵션)

쉽게 기억

ENTRYPOINT = 바꾸지 않을 실행기(java -jar app.jar), **CMD = 바꿀 수 있는 기본 옵션**. 스프링부트에선 보통 `ENTRYPOINT ["java","-jar","app.jar"]` 한 줄이면 충분합니다.

1-3. 스프링부트 Dockerfile 예제 (권장: 멀티스테이지)

멀티스테이지 빌드는 빌드용 이미지(JDK+Gradle)에서 jar를 만든 뒤, 그 **jar만** 가벼운 실행용 이미지(JRE)로 옮기는 방식입니다. 결과 이미지에 무거운 빌드 도구가 남지 않아 작고 안전합니다.

```
# syntax=docker/dockerfile:1

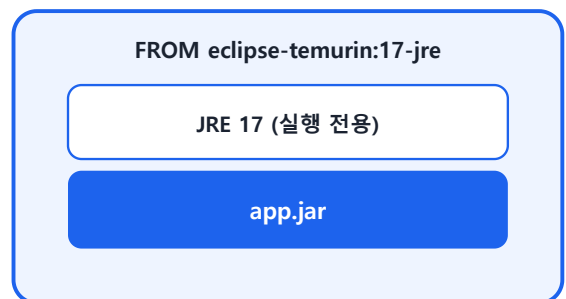
# ===== 1단계(빌드): 소스 -> jar =====
FROM eclipse-temurin:17-jdk AS build
WORKDIR /workspace
COPY . . # 소스 전체 복사(.dockerignore 제외 규칙 적용)
RUN chmod +x gradlew && ./gradlew clean bootJar --no-daemon

# ===== 2단계(런타임): jar만 담은 가벼운 최종 이미지 =====
FROM eclipse-temurin:17-jre AS runtime
WORKDIR /app
COPY --from=build /workspace/build/libs/*.jar app.jar # 1단계 산출물만 가져옴
# 보안: root 대신 전용 사용자로 실행. /app 소유권도 넘겨 파일 생성 가능하게
RUN groupadd --system spring && useradd --system --gid spring spring \
    && chown -R spring:spring /app
USER spring
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

1단계 · 빌드 스테이지 (무겁다)



2단계 · 런타임 (가볍다)



최종 이미지 ≈ 409MB

무거운 빌드 도구(JDK·Gradle·소스)는 최종 이미지에 남지 않는다

그림 2. 멀티스테이지: 빌드 도구는 버리고 jar만 남긴다 (최종 409MB)

실제로 겪은 함정

비루트(`USER spring`)로 바꾸면, 앱이 `/app` 아래에 파일(H2 파일·로그)을 만들 때 **권한 오류로 시작 이 실패**합니다. 그래서 `chown -R spring:spring /app` 로 소유권을 함께 넘겨야 합니다. (이 문서는 실제 실행 중 이 오류를 만나 반영했습니다.)

더 간단한 버전(로컬에서 이미 jar를 만든 경우)

```
# 먼저 ./gradlew bootJar 로 jar를 만들어 둔 뒤
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

짧지만, 빌드 환경(JDK·Gradle)이 **내 PC에 있어야** 합니다. 멀티스테이지는 "빌드까지 컨테이너 안에서" 하므로 어디서든 동일하게 재현됩니다.

1-4. .dockerignore — 빌드를 빠르고 안전하게

`COPY . .` 는 폴더 전체를 이미지로 보냅니다. 빌드 산출물·비밀파일까지 딸려가면 느리고 위험하니 제외 목록을 둡니다.

```
# .dockerignore
build/
.gradle/
data/
.git/
*.log
```

1-5. 빌드하고 실행하기

```
# 이미지 빌드 ( -t 이름:태그 . = 현재폴더의 Dockerfile )
docker build -t intro-jpa:1.0 .

# 실행 ( -p 호스트포트:컨테이너포트 )
docker run -p 8080:8080 intro-jpa:1.0

# 프로필/설정을 환경변수로 주며 실행 ( -e )
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=mysql intro-jpa:1.0
```

실무 팁

- **태그를 고정**하세요(`intro-jpa:1.0`). `latest` 만 쓰면 어떤 버전이 도는지 알기 어렵습니다.
- 자주 바뀌는 것(소스)은 **아래쪽**에 `COPY` → 위쪽 레이어가 캐시되어 재빌드가 빨라집니다.
- 베이스 이미지는 `-jre` (실행 전용)가 `-jdk` 보다 가볍습니다. `openjdk` 는 중단되어 `eclipse-temurin` 권장.

2 docker-compose — 여러 컨테이너를 한 번에

앱 하나만 띄울 땐 `docker run` 으로 충분합니다. 하지만 **앱 + DB**처럼 여러 컨테이너가 함께 떠야 하고, 서로 연결·순서·환경변수가 얹히면 명령이 길고 외우기 어렵습니다. 이걸 **YAML 파일 하나**에 적어 두고 `docker compose up` 한 방으로 띄우는 것이 Compose입니다.

2-1. 주요 키(key) 총정리

키	하는 일	예시
<code>services</code>	컨테이너 목록. 하위의 각 항목(이름)이 서비스 하나=컨테이너 하나	<code>services:</code> <code> app:</code> <code> db:</code>
<code>image</code>	기존 이미지를 그대로 사용	<code>image: mysql:8.4</code>
<code>build</code>	Dockerfile로 직접 빌드해서 사용	<code>build: .</code>
<code>ports</code>	호스트:컨테이너 포트 매핑	<code>"8080:8080"</code>
<code>environment</code>	컨테이너에 환경변수 주입	<code>SPRING_PROFILES_ACTIVE:</code> <code>mysql</code>
<code>env_file</code>	환경변수를 <code>.env</code> 파일에서 읽기	<code>env_file: .env</code>
<code>volumes</code>	데이터 영속화/폴더 마운트	<code>db-data:/var/lib/mysql</code>
<code>depends_on</code>	시작 순서 지정(+condition으로 상태 대기)	<code>db → app</code> 순서
<code>healthcheck</code>	서비스가 준비됐는지 검사하는 명령	<code>mysqladmin ping</code>
<code>restart</code>	재시작 정책	<code>on-failure</code> , <code>always</code>
<code>networks</code>	사용자 정의 네트워크(생략 시 자동 생성돼 서로 통신 가능)	보통 생략
<code>container_name</code>	컨테이너 이름 고정(선택)	<code>container_name: my-db</code>

version 키는 이제 안 씁니다

옛 예제의 첫 줄 `version: "3"` 은 최신 Compose에서 **불필요**(무시됨)합니다. 바로 `services:` 부터 쓰면 됩니다.

2-2. 스프링부트 + MySQL 예제

아래가 이 문서에서 **실제로 실행 검증한** compose 파일입니다. 앱(`app`)과 DB(`db`) 두 서비스로 이뤄집니다.

```

services:

# ----- 데이터베이스 -----
db:
  image: mysql:8.4                # 공식 MySQL 이미지(태그 고정)
  environment:                    # 첫 실행 때 DB·계정을 만들어 줌
    MYSQL_ROOT_PASSWORD: root1234
    MYSQL_DATABASE: introdb
    MYSQL_USER: intern
    MYSQL_PASSWORD: intern1234
  ports:
    - "3307:3306"                # 내 PC 3307 -> 컨테이너 3306 (GUI 접속용, 선택)
  volumes:
    - db-data:/var/lib/mysql     # 데이터 영속화
  healthcheck:                   # "접속 받을 준비 됐나?" 검사
    test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-proot1234"]
    interval: 5s
    timeout: 3s
    retries: 10

# ----- 스프링부트 앱 -----
app:
  build: .                        # 현재 폴더 Dockerfile로 빌드
  ports:
    - "8080:8080"
  environment:                   # application-mysql.properties 를 덮어씀
    SPRING_PROFILES_ACTIVE: mysql
    SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/introdb  # localhost 아님! 서비스명 db
    SPRING_DATASOURCE_USERNAME: intern
    SPRING_DATASOURCE_PASSWORD: intern1234
    SPRING_JPA_HIBERNATE_DDL_AUTO: update                # 데모용: 앱이 테이블 생성
  depends_on:
    db:
      condition: service_healthy  # db가 'healthy' 된 뒤 app 시작
  restart: on-failure

volumes:
  db-data:

```

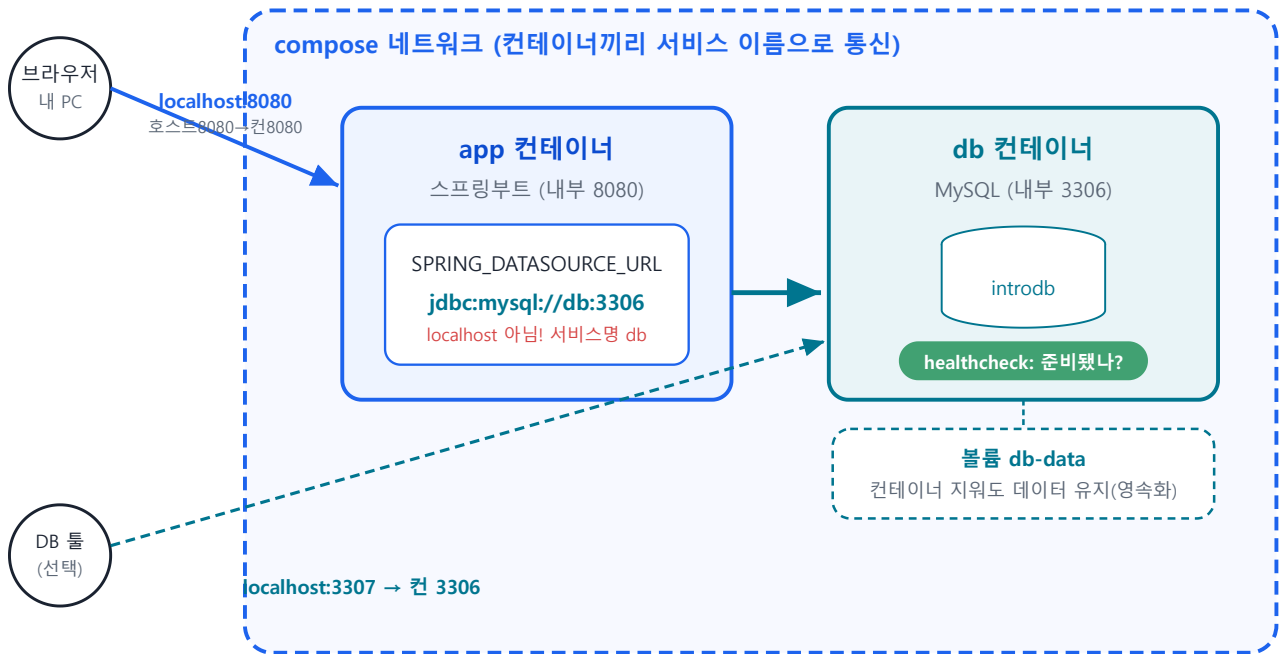



그림 3. app은 서비스 이름 db로 MySQL에 접속한다 (localhost가 아니다)

이 예제의 핵심 3가지

- **서비스 이름 = 호스트 이름.** app이 DB를 찾는 주소가 `jdbc:mysql://db:3306` — 같은 네트워크라 db로 통합니다.
- **환경변수로 프로필 전환.** `SPRING_PROFILES_ACTIVE=mysql` + `SPRING_DATASOURCE_*` 로 코드 수정 없이 MySQL에 붙습니다.
- **순서 보장.** `healthcheck` + `depends_on: condition: service_healthy` 로 "DB 준비 후 앱 시작"을 지킵니다.

2-3. 실행/종료 명령

```
docker compose up --build    # 빌드하며 스택 전체 실행(로그가 화면에 나옴)
docker compose up -d        # 백그라운드(-d)로 실행
docker compose ps            # 컨테이너 상태 확인
docker compose logs -f app   # app 로그 실시간 보기
docker compose down          # 컨테이너·네트워크 정리(볼륨은 유지)
docker compose down -v       # 볼륨(DB 데이터)까지 삭제
```

2-4. 자주 겪는 함정 & 체크리스트

#	함정	올바른 이해
1	앱에서 DB 주소를 <code>localhost:3306</code> 으로	컨테이너 안 localhost=자기 자신 . 다른 컨테이너는 서비스 이름 으로 → <code>db:3306</code>
2	<code>depends_on</code> 만 쓰면 DB가 준비 된 줄 안다	<code>depends_on</code> 은 시작만 보장. MySQL은 뜨는 데 시간이 걸림 → healthcheck + condition: service_healthy
3	포트를 <code>컨테이너:호스트</code> 순으 로 씀	항상 호스트:컨테이너 . <code>EXPOSE</code> 는 문서일 뿐 실제 개방이 아님
4	설정 키를 그대로 환경변수로	relaxed binding : <code>spring.datasource.url</code> → <code>SPRING_DATASOURCE_URL</code> (점·대시→_, 대문자)
5	볼륨 없이 DB 운영	컨테이너 삭제 시 데이터 증발 . 볼륨으로 영속화. <code>down -v</code> 는 볼륨까지 지우니 주의
6	비밀번호를 compose에 평문	학습용은 OK지만 실무는 .env·시크릿 으로 분리(.gitignore 필수)

한 장 요약 (치트시트)

Dockerfile 지시어

지시어	한 줄 요약
FROM	베이스 이미지(출발점)
WORKDIR	작업 폴더 지정
COPY	파일 복사(호스트→이미지)
RUN	빌드 중 명령 실행
ENV / ARG	환경변수(런타임) / 빌드변수(빌드 중)
EXPOSE	사용 포트 문서화(개방 아님)
ENTRYPOINT / CMD	항상 실행할 것 / 기본 인자
USER	실행 사용자(비루트 권장)

docker-compose 7|

키	한 줄 요약
services	컨테이너 목록
image / build	기존 이미지 사용 / Dockerfile로 빌드
ports	호스트:컨테이너 포트 매핑
environment	환경변수 주입(설정 됬어쓰기)
volumes	데이터 영속화
depends_on + healthcheck	시작 순서 + 준비 상태 대기
restart	재시작 정책

명령어 요약

```
docker build -t 이름:태그 .           # 이미지 빌드
docker run -p 8080:8080 이름:태그     # 컨테이너 실행
docker compose up --build              # 스택 전체 실행
docker compose down (-v)              # 정리 ( -v 는 볼륨까지 )
docker ps                             # 실행 중 컨테이너 / 이미지 목록
docker images                          # 이미지 목록
```

오늘 기억할 3가지

- ① **Dockerfile**은 위→아래 레시피. FROM으로 시작해 COPY·RUN으로 만들고 ENTRYPOINT로 실행. 스프링부트는 **멀티스테이지**로 작고 안전하게.
- ② **Compose**는 여러 컨테이너를 **YAML** 하나로. app + db를 up 한 번에. 접속은 **서비스 이름**으로.
- ③ **함정 1순위 = localhost**. 컨테이너 안 localhost는 자기 자신. 다른 컨테이너는 db:3306 처럼 서비스명으로.

이 문서의 모든 예제는 intro-jpa 프로젝트로 **실제 실행 검증**했습니다(멀티스테이지 빌드 성공, compose로 앱→MySQL 연결·HTTP 200 확인). 다만 Docker/이미지 버전은 계속 바뀌니 정확한 최신 정보는 공식 문서(docs.docker.com)를 확인하세요.

대상: 인턴·초급 개발자 · 실습 참고자료 · 기준 예제 intro-jpa (Spring Boot 4.1)